

50.003 - Software Testing

Learning Outcomes

By the end of this unit, you should be able to

1. Explain what is software testing
2. Articulate the purpose of software testing
3. List the components of a test
4. List the components of a test case
5. Develop simple testsuite for JavaScript programs using Jest

Software testing

Software testing is one of the methods to check and validate the produced software is behaving according to the specification.

Furthermore software testing helps to identify potential flaws in

- architectural design
- software security
- software performance

The following is a quote adopted from IBM <https://www.ibm.com/topics/software-testing>.

'Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do. The benefits of testing include preventing bugs, reducing development costs and improving performance.'

Why software testing

The answer this question. One way is to find out what if we do not conduct software testing? In particular for large scale mission critical systems, the absence of testing often leads to epic failures and excessive monetary cost.

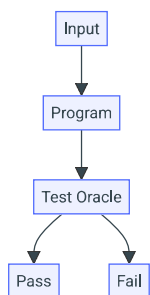
For instance in 1998, NASA launched a Mars Climate Orbiter. However due to an English Unit to Metric translation errors, the Orbiter lost contact after it passed behind Mars.

Anatomy of software testing

What is a test?

A test in a software system is an act to exercise software with test cases. A test have two goals

1. to find faults.
2. to show that the software behaves according to expectation (specification), i.e. to build confidence.



A test is a process of feeding the generated input to the test subject, i.e. the program, the output will be verified by a test oracle, which returns either `Pass` or `Fail` as result. The test oracle is often implemented as another program.

What is a test case?

A test case is a formal documentation of a test. A test case consists of

- an identifier (often linked to the user case identifier).
- a description of the purpose and the use case description being tested.
- a precondition
- a set of inputs
- the expected output
- the expected postcondition
- the execution history (the log)

Limitation of Software testing

Despite the importance of testing, testing alone is insufficient to ensure software free from bugs, since a failed test implies the existence of a software bug, a passed test implies that the software behaves according to the specification when given a particular instance of the input. But what about other input instances? What if the domain of the input is infinite.

"Testing shows the presence, not the absence of bugs" –Edsger Dijkstra

Testing a Node.js app

There are many tools for testing Node.js app. Jest is the one of the most popular option.

To add Jest to a Node.js project, we can run

```
1 npm i jest
```

in the project description file `package.json` change the follow line

```
1 "test": "echo \"Error: no test specified\" && exit 1"
```

to

```
1 "test": "jest"
```

For typescript project, we need to include `jest` in the `types` field in `tsconfig.json`.

```

1  {
2    "compilerOptions": {
3      "target": "ES2020",
4      "module": "ES2020",
5      "lib": ["ES2020"],
6      "outDir": "./dist",
7      "rootDir": "./",
8      "strict": true,
9      "esModuleInterop": true,
10     "skipLibCheck": true,
11     "forceConsistentCasingInFileNames": true,
12     "moduleResolution": "node",
13     "types": ["jest", "node"],
14     "isolatedModules": true
15   }
16 }

```

Consider in the `src/mymath.ts`

```

1  function sum(x: number, y: number): number {
2    return x + y;
3  }
4  export = {sum}

```

In the `test/mymath.test.ts`

```

1  import { sum } from '../src/mymath';
2
3  describe("mymath sum tests", () => {
4    test ("summing of two positive numbers", () => {
5      const result = sum(1,2);
6      expect(result).toBe(3);
7    })
8    test ("summing of two negative numbers", () => {
9      const result = sum(-3,-2);
10     expect(result).toBe(-5);
11   })
12 })

```

The `describe()` function defines a set of tests should be logically grouped together as a test suite. In the above case, in the test suite consists of two tests. Each test is define by a call to `test()` function, which expect a description for the test, the action test function. In the test function, we compute the actual result by calling the test subject, `sum()`, and compares it with the expected result.

To execute the test, we run

```

1  npm test

```

which runs the above test suite against the test subject and generates the following report in the console.

```

1  > my_test_app@1.0.0 test
2  > jest
3
4  PASS  test/mymath.test.ts
5  mymath sum tests
6    ✓ summing of two positive numbers (12 ms)
7    ✓ summing of two negative numbers (3 ms)
8
9  Test Suites: 1 passed, 1 total
10 Tests:      2 passed, 2 total
11 Snapshots:  0 total
12 Time:       0.35 s, estimated 1 s
13 Ran all test suites.

```

Testing OOP

We can use Jest to test an object instance of a class, consider the following file `src/FibSeq.ts`

```

1  export class FibSeq {
2    prev = 0;
3    curr = 1;
4
5    next(): number {
6      const res = this.prev + this.curr;
7      this.prev = this.curr;
8      this.curr = res;
9      return res;
10   }
11 }

```

We can test it with the following test suite

```

1  import { FibSeq } from '../src/FibSeq';
2
3  describe("FibSeq class test", () => {
4    const fibSeq = new FibSeq();
5    test ("first fib num is 1", () => {
6      const result = fibSeq.next();
7      expect(result).toBe(1);
8    });
9    test ("second fib num is 2", () => {
10     const result = fibSeq.next();
11     expect(result).toBe(2);
12   });
13 })

```

Which will run the two tests in sequence.

If we want to have the object being reset for each test case.

```

1  describe("FibSeq class test with setup and tear down", () =>{
2    let fibSeq: FibSeq | null = null;
3    beforeEach(() => {
4      fibSeq = new FibSeq();
5    });
6    test ("first fib num is 1 after reset", () => {
7      const result = fibSeq?.next();
8      expect(result).toBe(1);
9    });
10   test ("second fib num is 1 after reset", () => {
11     const result = fibSeq?.next();
12     expect(result).toBe(1);
13   });
14   afterEach(() => {
15     fibSeq = null;
16   })
17 })

```

Further Readings

- <https://www.lambdatest.com/jest>